

Native Mobile Development Learning Plan

Kotlin + Swift

Team: CarMDConnect

Program start: Monday, 14 September 2026

Important — do NOT write the demo code using AI. Every part must be written by hand. The whole point of this project is hands-on native understanding; letting an AI generate the implementation defeats the goal. AI and docs may be used to look up concepts and APIs only — not to produce the demo code.

How to read this plan: *Part I — Knowledge* is the reference — the concepts and the components the demo must contain. *Part II — Weekly Schedule* is where you actually build the **Native Device Monitor Demo**, one working slice per week — see the **Demo milestone** line under each week. The *Appendix* is Ms. Hieu's separate plugin track.

Table of Contents

PART I — Knowledge

1. Goal
2. Final Learning Project
3. Learning Scope
4. Native UI Requirements
5. Permission Manager
6. Bluetooth BLE Manager
7. Camera and Media Manager
8. API Client
9. Background Device Monitor
10. Native Architecture
11. Android Architecture
12. iOS Architecture
13. Public Service Interfaces
14. What Is Explicitly Out of Scope
15. Final Learning Outcome

PART II — Weekly Schedule

1. Week 1 — Fundamentals and UI
2. Week 2 — Native APIs and BLE
3. Week 3 — NativeCore and Background Monitor
4. Final Half Week — Testing and Cleanup
5. Total Time
6. Priority Order

APPENDIX — Independent Track (Ms. Nguyen Thị Thu Hieu)

Daily plan (Day 1–5)

A.1 Why this track is different

A.2 Reverse Research — Understand the Ionic/App Side

A.3 Phase 1 — Verify TripTracker Is Truly Standalone

A.4 Phase 2 — Standalone Plugin Test Harness (before integrating into the app)

PART I — Knowledge

1. Goal

Build practical native mobile development skills for:

- Reading existing Android and iOS native code.
- Understanding native application lifecycle.
- Modifying existing native code.
- Using native device APIs.
- Building reusable native services/classes.
- Working with asynchronous native APIs.
- Working with Bluetooth BLE.
- Understanding background execution.
- Building a small native application on Android and iOS.

The main implementation languages are:

- Android: Kotlin
- iOS: Swift

Java is only required as a code-reading skill for existing Android code.

Capacitor plugin development is out of scope.

2. Final Learning Project

Project Name

Native Device Monitor Demo

Objective

Build a small native application for Android and iOS that demonstrates several important native mobile capabilities.

The application must provide:

1. Basic native UI
2. Permission management
3. Bluetooth BLE communication
4. Camera and media management
5. HTTP API communication
6. Background device monitoring

The application should use reusable native services instead of putting device logic directly inside the UI.

3. Learning Scope

The project focuses on these concepts:

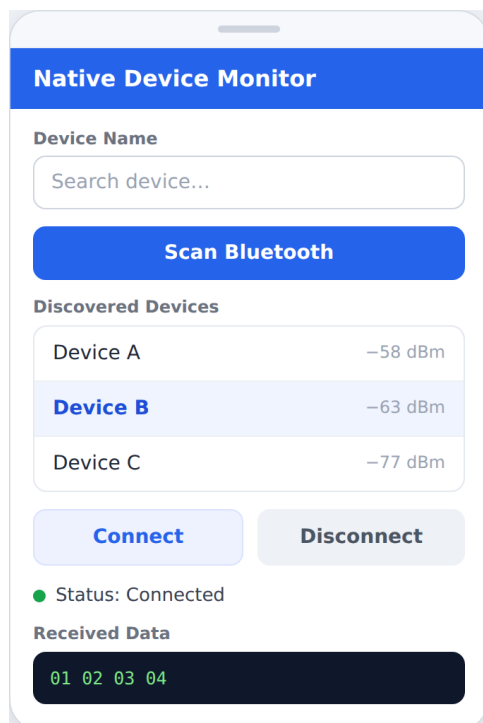
Native UI
↓
Native Service
↓
Platform API
↓
Device / System

The developer should understand how to:

- Call native APIs.
- Handle asynchronous operations.
- Handle permissions.
- Handle lifecycle changes.
- Handle callbacks/events.
- Handle errors.
- Separate UI from native functionality.
- Build reusable native classes/services.

4. Native UI Requirements

Suggested UI — main screen (BLE + status + received data):



The UI does not need to be visually sophisticated.

However, the developer must become comfortable with common native UI components.

The application should use at least:

- Text label
- TextBox / TextField
- Button
- List / ListView
- Image
- Basic navigation or screen/section switching
- Loading/progress indicator
- Basic status/error message

The UI should be sufficient to operate and test all demo features.

Example:

Native Device Monitor

Device Name:

[_____]

[Scan Bluetooth]

Discovered Devices:

Device A

Device B

Device C

[Connect]

Status: Connected

Received Data:

01 02 03 04

[Disconnect]

The UI should remain simple.

Do not spend time on:

- Advanced UI design
- Animations

- Complex layouts
- Design systems
- Advanced navigation architecture
- Production-level UI architecture

5. Permission Manager

Create a reusable native permission service.

Responsibilities:

- Check permission status.
- Request permission.
- Handle granted state.
- Handle denied state.
- Handle permanently denied/restricted state where applicable.
- Support permissions required by Bluetooth and camera/media features.

Example API:

```
checkPermission()  
requestPermission()  
checkMultiplePermissions()  
requestMultiplePermissions()
```

The UI should call the service instead of directly implementing permission logic.

6. Bluetooth BLE Manager

Bluetooth is the most important device feature in this project.

The implementation must cover the complete basic BLE workflow.

Required flow:

```
Scan  
↓  
Select Device  
↓  
Connect  
↓  
Discover Services  
↓  
Discover Characteristics  
↓  
Subscribe to Notification  
↓  
Receive Data  
↓  
Read Data
```



Disconnect

6.1 Scan

The application must:

- Check Bluetooth availability.
- Check required permissions.
- Start BLE scan.
- Receive discovered devices.
- Display discovered devices in a list.
- Stop scanning.
- Handle scan errors.

Example API:

```
isAvailable()  
isEnabled()  
startScan()  
stopScan()  
getDiscoveredDevices()
```

6.2 Connect

The application must:

- Select a device from the list.
- Connect to the device.
- Report connection state.
- Handle connection failure.
- Handle unexpected disconnection.

Example API:

```
connect(device)  
disconnect()  
getConnectionState()
```

6.3 Service and Characteristic Discovery

After connection:

- Discover BLE services.
- Identify characteristics.
- Identify readable characteristics.
- Identify notification-capable characteristics.

The implementation should demonstrate understanding of:

- Service
- Characteristic
- UUID
- Read
- Write
- Notify / Indicate

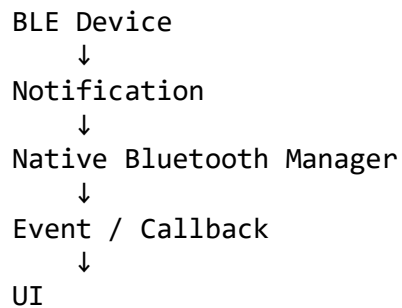
6.4 Notification

The application must subscribe to a notification-capable characteristic.

Example:

```
subscribe(characteristic)
```

When the device sends data:



The UI should display received data.

6.5 Read / Receive Data

The application must support reading data from a readable characteristic.

Example API:

```
read(characteristic)
```

The application should display the received value.

The implementation should demonstrate:

- Byte data handling.
- Converting bytes to a useful representation.
- Async result handling.
- Read errors.

6.6 Disconnect

The application must:

- Disconnect manually.

- Detect unexpected disconnection.
- Update UI state.
- Clean up notification subscriptions/resources where required.

Example API:

`disconnect()`

6.7 Bluetooth Scope Limitations

The project does NOT require:

- Complex BLE protocol implementation.
- Production device protocol.
- OTA firmware update.
- Complex GATT abstraction.
- Advanced pairing/security workflows.
- Bluetooth Classic support unless required by a specific test device.

The goal is to understand the complete BLE lifecycle and data flow.

7. Camera and Media Manager

Implement a reusable native media service.

Required functionality:

- Request camera permission.
- Open native camera.
- Capture an image.
- Select an image from media library.
- Return image/media information.
- Display the selected image in the UI.

Example API:

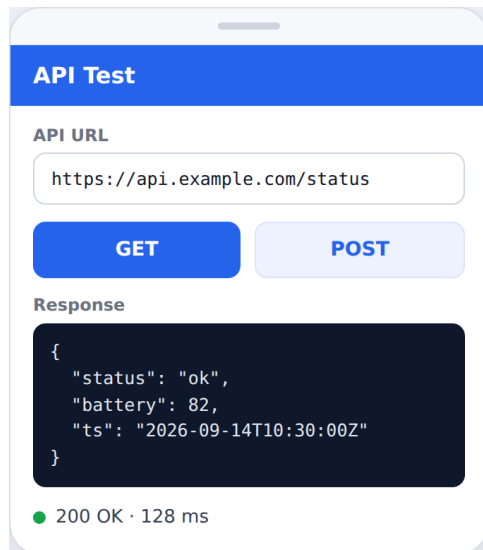
```
requestCameraPermission()
capturePhoto()
pickMedia()
getMediaInfo()
```

Do not implement:

- Image processing.
- Video editing.
- Media compression pipeline.
- Advanced media management.

8. API Client

Suggested UI — API test screen:



Implement a small reusable native HTTP client.

Required functionality:

- GET request.
- POST request.
- Headers.
- Parameters.
- JSON request/response.
- HTTP status handling.
- Network error handling.
- Timeout handling.
- Async request handling.

Example API:

```
get(url)
post(url, body)
```

The UI should provide a simple test interface.

Example:

API URL :
[_____]

[GET]

Response:

```
-----  
{  
  ...  
}  
-----
```

Do not implement:

- Authentication system.
- Token refresh.
- Retry framework.
- Caching.
- Pagination.
- Complex API architecture.

9. Background Device Monitor

Implement a native background monitoring service.

The service periodically collects:

- Battery percentage.
- Wi-Fi/network status.
- Cellular/network availability where available.
- Bluetooth enabled status.
- Bluetooth connection status where available.
- Timestamp.

Example record:

```
{  
  "timestamp": "2026-09-11T10:30:00",  
  "battery": 82,  
  "wifiEnabled": true,  
  "wifiConnected": true,  
  "cellularAvailable": true,  
  "bluetoothEnabled": true,  
  "bluetoothConnected": false  
}
```

Required API

```
start(interval)  
stop()  
getStatus()  
getLogs()  
clearLogs()
```

Local Logging

Use a simple local file or equivalent storage mechanism.

JSONL is sufficient.

Example:

```
{ "timestamp": "...", "battery": 82, "wifiConnected": true }
{ "timestamp": "...", "battery": 81, "wifiConnected": true }
{ "timestamp": "...", "battery": 81, "wifiConnected": false }
```

Background Execution

The implementation must demonstrate:

- Foreground execution.
- Background execution.
- Returning to foreground.
- OS termination.
- Force-kill behavior.

The goal is to understand the actual platform behavior.

Do not attempt to bypass platform restrictions.

If continuous execution after force-kill is not supported, document the behavior through code comments or learning notes and implement the platform-supported mechanism.

10. Native Architecture

Use a simple reusable service architecture.

```
UI
|
+-- PermissionManager
+-- BluetoothManager
+-- MediaManager
+-- ApiClient
+-- BackgroundMonitor
    |
    +-- BatteryProvider
    +-- NetworkProvider
    +-- BluetoothProvider
    +-- FileLogger
|
+-- Platform APIs
```

Important rule:

UI != Device Logic

The UI should:

- Receive user input.
- Call service APIs.
- Display results.
- Display errors/status.

The service should:

- Call native APIs.
- Handle callbacks/events.
- Handle permissions.
- Handle errors.
- Manage device state.

11. Android Architecture

Use Kotlin.

Possible structure:

```
android/  
├── ui/  
├── nativecore/  
│   ├── permission/  
│   ├── bluetooth/  
│   ├── media/  
│   ├── network/  
│   ├── background/  
│   ├── device/  
│   └── logging/  
└── ...
```

The exact package structure can be simpler if necessary.

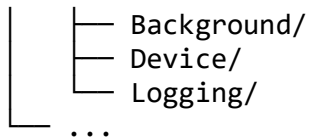
Do not introduce complex architecture frameworks.

12. iOS Architecture

Use Swift.

Possible structure:

```
ios/  
├── UI/  
├── NativeCore/  
│   ├── Permission/  
│   ├── Bluetooth/  
│   ├── Media/  
│   └── Network/
```



Use native Apple frameworks.

Do not introduce complex architecture frameworks.

13. Public Service Interfaces

The exact syntax can differ between Kotlin and Swift.

The important goal is to keep the public API simple.

PermissionManager

```
checkPermission()  
requestPermission()
```

BluetoothManager

```
startScan()  
stopScan()
```

```
connect(device)  
disconnect()
```

```
discoverServices()
```

```
read(characteristic)  
subscribe(characteristic)  
unsubscribe(characteristic)
```

```
getConnectionState()
```

MediaManager

```
capturePhoto()  
pickMedia()  
getMediaInfo()
```

ApiClient

```
get()  
post()
```

BackgroundMonitor

```
start()  
stop()  
getStatus()  
getLogs()  
clearLogs()
```

14. What Is Explicitly Out of Scope

Do NOT spend time on:

- Capacitor plugin implementation.
- Ionic integration.
- Angular integration.
- Advanced Kotlin.
- Advanced Swift.
- Java application development.
- Advanced UI/UX.
- UI design system.
- Advanced architecture frameworks.
- Dependency injection frameworks.
- Unit testing frameworks.
- CI/CD.
- App Store deployment.
- Play Store deployment.
- Authentication.
- Database.
- Cloud backend.
- Complex BLE protocols.
- BLE firmware update.
- Bluetooth Classic.
- Advanced media processing.
- Production-grade API framework.
- Production-grade logging framework.
- Project documentation / README.

15. Final Learning Outcome

After completing the project, the developer should be able to:

Read
↓
Understand
↓
Modify
↓
Implement
↓
Test
↓
Debug

basic Android and iOS native functionality.

The developer should also understand this architecture:

```
Native UI
  ↓
Native Service
  ↓
Platform API
  ↓
Device
```

And specifically understand the complete BLE flow:

```
Scan
  ↓
Connect
  ↓
Discover Services
  ↓
Discover Characteristics
  ↓
Subscribe Notification
  ↓
Receive Data
  ↓
Read Data
  ↓
Disconnect
```

The next optional learning phase can later expose the same NativeCore through a Capacitor plugin:

```
Ionic
  ↓
Capacitor Plugin
  ↓
NativeCore
  ↓
Android / iOS APIs
```

PART II — Weekly Schedule

1. Week 1 — Fundamentals and UI

Demo milestone — by end of this week: a running app **shell** on Android + iOS (text input, button, list, image, status, async update) — still no device features.

Suggested target — by end of this week:

- App builds and runs on both Android and an iOS simulator.
- Screen shows text input, button, list, image and status text.
- Button updates the list; an async result updates the status text without freezing the UI.
- No device features required yet.

Suggested daily plan — Mon 14 Sep → Fri 18 Sep (Day 1–5)

- **Day 1 — Mon 14 Sep — Kotlin fundamentals**
 - *Do:* variables, functions, classes/interfaces, null safety, collections, lambdas, the idea of coroutines
 - *Exercise:* small Kotlin program — a data class `Device(name, rssi)`; filter a `List<Device>` with a lambda; print with null-safe access
 - *Target:* read Kotlin comfortably; explain null safety; write a lambda unaided
- **Day 2 — Tue 15 Sep — Swift fundamentals**
 - *Do:* variables, functions, class vs struct, protocols, optionals, closures, basic `async/await`
 - *Exercise:* Swift playground — a struct `Device`; map/filter an array with a closure; unwrap optionals safely
 - *Target:* read Swift comfortably; explain optionals and struct-vs-class; write a closure unaided
- **Day 3 — Wed 16 Sep — Android fundamentals + Java reading**
 - *Do:* project structure, Activity, Context, Intent, Manifest, lifecycle, Gradle basics; recognise Java class/interface/override/callback
 - *Exercise:* empty Android (Kotlin) app that logs every lifecycle callback (`onCreate`→`onDestroy`); read one Java file and label its parts
 - *Target:* app runs; can point to each lifecycle log; can read Java without writing it
- **Day 4 — Thu 17 Sep — iOS fundamentals + start Native UI on Android**
 - *Do:* Xcode structure, ViewController, app lifecycle, frameworks, Info.plist; Android views (`TextView`/`EditText`/`Button`/`RecyclerView`/`ImageView`/`progress`)
 - *Exercise:* empty iOS app logging app lifecycle; on Android lay out a label + text field + button + list
 - *Target:* both apps run; the Android screen shows the basic controls
- **Day 5 — Fri 18 Sep — Native UI — finish Android + iOS; deliver Week 1 app**

- *Do:* finish Android UI and build the same iOS UI (label/textfield/button/list/image/progress); update UI from an async result
- *Exercise:* button appends the text-field value to the list; show a spinner ~1 s then a status text via an async callback
- *Target:* Week 1 deliverable: Android + iOS app with text input, button, list, image, status text, async update — no device features yet

Estimated time: ~17 hours

Kotlin — ~3 hours

Learn only the Kotlin required for Android development:

- Variables
- Functions
- Classes
- Interfaces
- Null safety
- Collections
- Lambda
- Basic extension functions
- Basic async concepts

Do not study advanced Kotlin.

Swift — ~3 hours

Learn:

- Variables
- Functions
- Classes
- Structs
- Protocols
- Optionals
- Closures
- Basic async/await

Android Fundamentals — ~2 hours

Learn:

- Android project structure.
- Activity.
- Context.
- Intent.

- Manifest.
- Lifecycle.
- Gradle basics.

iOS Fundamentals — ~2 hours

Learn:

- Xcode project structure.
- ViewController.
- App lifecycle.
- Frameworks.
- Info.plist.

Java Reading — ~1 hour

Only learn enough to recognize:

- Class
- Interface
- Method
- Constructor
- Field
- Override
- Generic
- Callback

Goal:

Read existing Java Android code.

Do not build a Java project.

Native UI — ~6 hours

Android:

- TextView
- EditText
- Button
- RecyclerView/ListView
- ImageView
- Progress indicator
- Basic layout
- Click listeners
- Updating UI from async results

iOS:

- Label
- TextField
- Button
- List
- Image
- Progress indicator
- Basic layout
- Event handling
- Updating UI from async results

Week 1 Deliverable

A simple native application on Android and iOS with:

- Text input.
- Button.
- List.
- Image.
- Status text.
- Basic async result display.

No device functionality is required yet.

2. Week 2 — Native APIs and BLE

Demo milestone — by end of this week: device features working — permissions, the full BLE flow (scan → connect → discover → notify → read → disconnect), camera/media, and API GET/POST.

Suggested target — by end of this week:

- Permissions requested and handled on both platforms (granted **and** denied paths).
- Full BLE flow works against a test peripheral: scan → connect → discover → notify → read → disconnect, with the UI reflecting each state.
- Unexpected disconnect is detected and the UI recovers.
- Camera capture + library pick show an image; API GET and POST show success / error / timeout.

Suggested daily plan — Mon 21 Sep → Fri 25 Sep (Day 1–5)

- **Day 1 — Mon 21 Sep — Permissions + BLE concepts & Step 1**
 - *Do:* permission check/request/result; BLE central/peripheral, service, characteristic, UUID, read/write/notify
 - *Exercise:* request BLE + camera permissions on both platforms and handle granted/denied; write a short note on the BLE object model

- *Target:* permissions handled including the denied path; can explain service/characteristic/notify
- **Day 2 — Tue 22 Sep — BLE scan + connect — Steps 2-3**
 - *Do:* startScan/stopScan, list discovered devices, connect/disconnect, report connection state
 - *Exercise:* scan and show devices in a list; connect to a chosen device (nRF Connect or an ESP32 as the test peripheral); show Connected/Disconnected
 - *Target:* can scan, pick, connect and see live connection state on both platforms
- **Day 3 — Wed 23 Sep — BLE discovery + read — Steps 4-6**
 - *Do:* discover services & characteristics, identify readable/notify ones, read a characteristic, convert bytes→value
 - *Exercise:* after connect, list services/characteristics; read one characteristic and print raw bytes + a decoded value
 - *Target:* can locate a target characteristic and read + decode its value
- **Day 4 — Thu 24 Sep — BLE notifications, disconnect & errors — Steps 7-9 + full-flow test**
 - *Do:* subscribe to a notify characteristic, route data → event → UI, handle manual + unexpected disconnect, clean up subscriptions
 - *Exercise:* stream notification values into the UI; kill the peripheral to force an unexpected disconnect and update state
 - *Target:* full BLE flow works scan→...→disconnect; UI reflects every state including errors
- **Day 5 — Fri 25 Sep — Camera/Media + Networking**
 - *Do:* camera permission, capture photo, pick from library, show image; GET/POST, JSON, headers, timeout, async, error handling
 - *Exercise:* capture + pick an image and display it; one screen that GETs a URL and shows JSON, plus a POST with a JSON body; force a timeout
 - *Target:* media capture/pick works; GET & POST work with visible success / error / timeout handling

Estimated time: ~17 hours

Permissions — ~2 hours

Implement:

- Permission checking.
- Permission requests.
- Permission result handling.

Focus on permissions required by:

- Bluetooth.
- Camera.

- Media.

Bluetooth BLE — ~10 hours

This is the largest learning block.

Step 1

BLE concepts:

- Central/peripheral.
- Service.
- Characteristic.
- UUID.
- Read.
- Write.
- Notify.
- Connection state.

Step 2

Implement scan:

```
startScan()  
stopScan()
```

Step 3

Implement connection:

```
connect()  
disconnect()
```

Step 4

Implement service discovery.

Step 5

Implement characteristic discovery.

Step 6

Implement read.

Step 7

Implement notification subscription.

Step 8

Handle received notification data.

Step 9

Handle disconnect and errors.

Test the complete flow:

```
Scan
↓
Select device
↓
Connect
↓
Discover services
↓
Find characteristic
↓
Subscribe notification
↓
Receive data
↓
Read data
↓
Disconnect
```

Camera / Media — ~2 hours

Implement:

- Camera permission.
- Capture photo.
- Select media.
- Display image.

Networking — ~3 hours

Implement:

- GET.
- POST.
- JSON.
- Error handling.
- Timeout.
- Async response.

3. Week 3 — NativeCore and Background Monitor

Demo milestone — by end of this week: the app **refactored into NativeCore services** (Permission / Bluetooth / Media / Api) plus a **BackgroundMonitor** logging device status; the UI drives every feature.

Suggested target — by end of this week:

- Device logic lives only in services (Permission / Bluetooth / Media / Api); the UI just calls them.
- Providers return correct battery / network / bluetooth status; FileLogger appends, reads and clears JSONL.
- BackgroundMonitor logs a status record on an interval in the foreground and stops cleanly.
- Background behaviour documented for foreground / background / return / OS-kill / force-kill on both platforms (incl. the iOS interval caveat).

Suggested daily plan — Mon 28 Sep → Fri 2 Oct (Day 1–5)

- **Day 1 — Mon 28 Sep — NativeCore refactoring**
 - *Do:* split existing code into PermissionManager, BluetoothManager, MediaManager, ApiClient; UI only consumes services
 - *Exercise:* move all device logic out of the UI into the four services; UI calls service APIs only
 - *Target:* no device logic left in the UI; each service exposes a clear public API
- **Day 2 — Tue 29 Sep — Device providers + start FileLogger**
 - *Do:* BatteryProvider, NetworkProvider, BluetoothProvider; FileLogger append
 - *Exercise:* each provider returns the current status; FileLogger appends a JSONL line
 - *Target:* providers return correct values; a log line is written to file
- **Day 3 — Wed 30 Sep — FileLogger finish + BackgroundMonitor start**
 - *Do:* read/clear logs + file-error handling; BackgroundMonitor start/stop/getStatus with a collect→record→save loop
 - *Exercise:* implement start(interval) that writes a status record each tick; add getLogs/clearLogs
 - *Target:* monitor writes periodic records in the foreground; logs are readable and clearable
- **Day 4 — Thu 1 Oct — BackgroundMonitor behaviour tests**
 - *Do:* run in foreground, background, return, OS termination, force-kill; use the platform-correct mechanism (Android foreground service / WorkManager; iOS BGTaskScheduler / background modes)
 - *Exercise:* observe and note what happens to the interval in each state; document that iOS does not guarantee a fixed interval
 - *Target:* a written behaviour table (Android vs iOS) for every app state; no attempt to bypass OS limits
- **Day 5 — Fri 2 Oct — Native UI integration + buffer**
 - *Do:* wire the UI to PermissionManager / BluetoothManager / MediaManager / ApiClient / BackgroundMonitor

- *Exercise:* one screen can operate all major features (scan/connect, capture, GET/POST, start/stop monitor)
- *Target:* every feature is operable from the UI

Estimated time: ~14 hours

NativeCore Refactoring — ~3 hours

Separate the implementation into:

PermissionManager
BluetoothManager
MediaManager
ApiClient

The UI should only consume these services.

Device Providers — ~2 hours

Implement:

BatteryProvider
NetworkProvider
BluetoothProvider

FileLogger — ~2 hours

Implement:

- Append log.
- Read logs.
- Clear logs.
- Handle file errors.

BackgroundMonitor — ~5 hours

Implement:

start()
stop()
getStatus()
getLogs()

The monitor should:

1. Start background monitoring.
2. Collect device status.
3. Create a log record.
4. Save the record.
5. Repeat according to the selected interval.
6. Stop cleanly.

Test:

- Foreground.
- Background.
- Return to foreground.
- OS termination.
- Force-kill.

Native UI Integration — ~2 hours

Connect the UI to:

- PermissionManager.
- BluetoothManager.
- MediaManager.
- ApiClient.
- BackgroundMonitor.

The UI must allow the user to operate all major features.

4. Final Half Week — Testing and Cleanup

Demo milestone — by end of this stage: the **finished Native Device Monitor Demo** — every feature tested end-to-end and the code cleaned up.

Suggested target — by end of this stage:

- Every feature passes the end-to-end checklist (Permission / BLE / Camera / API / Background), or defects are logged.
- Refactor checklist green: no device logic in UI, errors handled, resources released, BLE state consistent, monitor stoppable, no dead deps or duplicates.
- The Native Device Monitor Demo runs end-to-end on both platforms.

Suggested daily plan — Mon 5 Oct → Tue 6 Oct (Day 1–2)

- **Day 1 — Mon 5 Oct — End-to-end testing**
 - *Do:* test Permission (check/request/deny/retry), BLE (scan→...→reconnect), Camera, API (GET/POST/success/error/timeout), Background (start/bg/return/stop/restart/OS-kill/force-kill)
 - *Exercise:* run each checklist path and log every defect
 - *Target:* all flows pass, or the defects are recorded
- **Day 2 — Tue 6 Oct — Refactoring + cleanup**
 - *Do:* verify UI has no device logic, services have clear responsibilities, async/errors handled, resources released, BLE state consistent, monitor stoppable, no dead deps or duplicated logic
 - *Exercise:* run the refactor checklist and fix the findings
 - *Target:* checklist all green; the demo build is clean

Estimated time: ~8 hours

End-to-End Testing — ~4 hours

Test the complete demo.

Permission

Check
Request
Deny
Retry

Bluetooth

Scan
Connect
Discover
Subscribe
Receive notification
Read
Disconnect
Reconnect

Camera

Request permission
Capture
Select
Display

API

GET
POST
Success
Error
Timeout

Background Monitor

Start
Background
Return
Stop
Restart
OS termination
Force-kill

Refactoring — ~4 hours

Check:

- UI does not contain device logic.
- Services have clear responsibilities.

- Async callbacks are handled correctly.
- Errors are handled.
- Resources are released correctly.
- Bluetooth connection state is consistent.
- Background monitoring can be stopped.
- No unnecessary dependencies.
- No duplicated feature logic.

5. Total Time

Starting **Monday, 14 September 2026**, running Mon–Fri at ~3 h/day, the plan finishes around **Tuesday, 6 October 2026**.

Target:

~56 hours

Suggested distribution:

Week 1	~17 h
Week 2	~17 h
Week 3	~14 h
Final half	~8 h

Total	~56 h

At approximately:

2.5–3 hours/day

the project fits into approximately:

3.5 weeks

6. Priority Order

If time becomes limited, use this priority order:

P0 – Must complete

1. Kotlin fundamentals
2. Swift fundamentals
3. Native UI basics
4. Permission
5. BLE scan
6. BLE connect
7. BLE service/characteristic discovery
8. BLE notification
9. BLE read/receive
10. BLE disconnect

11. Background Monitor

P1 – Should complete

- 12. Camera
- 13. Media picker
- 14. API GET
- 15. API POST
- 16. File logging

P2 – Can simplify if necessary

- 17. Advanced API error handling
- 18. Advanced media handling
- 19. Complex UI
- 20. Additional lifecycle experiments

APPENDIX — Independent Track (For Ms. Nguyen Thị Thu Hieu)

Suggested target — by end of the week:

- Reverse-research notes done: screen map, data-sharing map, and how the app calls the plugin.
- Phase 1 decoupling report delivered: call-site inventory (guarded/unguarded) + shared-storage map + uninstall-test pass/fail; the app degrades gracefully with TripTracker removed.
- Phase 2 harness runs the released plugin end-to-end (Start/Pause/End Trip + streaming log) before any integration.

Daily plan for Ms. Hieu — Mon 14 Sep → Fri 18 Sep (Day 1–5)

- **Day 1 — Mon 14 Sep — Reverse research — screen map + how the app calls the plugin**
 - *Do:* map Angular pages/routes under `src/app`; find which screens touch TripTracker; read how the app imports and calls TripTracker (methods + `addListener`)
 - *Exercise:* from one screen, trigger a TripTracker call via a button and log the response; also call it in `ngOnInit` and read `console.log` (Safari Web Inspector / `chrome://inspect`)
 - *Target:* can name every screen that uses TripTracker and show one working call + one listener event in the console
- **Day 2 — Tue 15 Sep — Reverse research — data-sharing surfaces**
 - *Do:* identify all data-sharing points (Capacitor Preferences → `UserDefaults/SharedPreferences`, App Group, shared SQLite/files, event bus) and classify each as code-dependency vs data-contract
 - *Exercise:* produce a one-page shared-storage map: what writes it, what reads it, iOS vs Android backing store
 - *Target:* a written data-sharing map with every surface classified
- **Day 3 — Wed 16 Sep — Phase 1 — static / code-level decoupling audit**
 - *Do:* check native TripTracker imports no app/other-plugin classes; confirm it ships its own `.podspec` / `build.gradle`; confirm the app imports only the public API; list permission/background-mode entries in app vs plugin
 - *Exercise:* grep every TripTracker call site in the app and mark each guarded / unguarded (`isPluginAvailable` / `try-catch`)
 - *Target:* call-site inventory complete; every unguarded call flagged
- **Day 4 — Thu 17 Sep — Phase 1 — runtime removal test + data-contract + report**
 - *Do:* run `npm uninstall <plugin> && npx cap sync`; launch the app; confirm every screen loads without crash; verify app reads tolerate a missing key
 - *Exercise:* capture before/after; fix or document any UNIMPLEMENTED crash by guarding the call

- *Target:* decoupling report delivered — call-site inventory + shared-storage map + pass/fail of the removal test
- **Day 5 — Fri 18 Sep — Phase 2 — standalone test harness**
 - *Do:* build a minimal Capacitor shell app; install the released package (or the npm pack tarball); wire Start/Pause/End Trip + Token/Vehicle ID + a streaming log via `addListener`
 - *Exercise:* run a full trip start → events → end in the harness and watch the foreground/background transitions
 - *Target:* harness runs the plugin end-to-end from the packaged release before touching the real app; ping Bảo to finalise the inputs

A.1 Why this track is different

Ms. Hieu is already strong in native, so she runs independently from the from-scratch learners above. Her direction is the reverse: instead of learning native to reach the app, she learns just enough of the Ionic/Capacitor app to (a) understand how the app consumes native, and (b) harden TripTracker into a clean, standalone plugin package.

A.2 Reverse Research — Understand the Ionic/App Side

Target: basic working knowledge, not app development skills.

1. Screen map — where the app's screens live
 - Angular pages/components under `src/app/...`; routing in the routing module(s).
 - Identify which screens actually touch TripTracker.
2. Data-sharing surfaces between app and native / TripTracker
 - Capacitor Preferences → backed by `UserDefaults` (iOS) / `SharedPreferences` (Android)
 - iOS App Group container, if trip data is shared across process/extension
 - Shared SQLite / files, if any
 - Event bus: plugin → JS via `addListener` (native `notifyListeners`) For each surface, classify it: is it a CODE dependency, or just a DATA contract?
3. How the app calls the plugin
 - Request/response (public API):


```
import { TripTracker } from '<plugin-package>';
const res = await TripTracker.startTrip({ userToken, vehicleId });
```
 - Streaming/events (trip tracking is event-driven — this matters MORE than req/res):


```
const h = await TripTracker.addListener('tripUpdate', d => console.log(d));
```
 - Two quick ways to exercise it:

- a) A button in HTML → call the method → render res on screen.
- b) Call it in the page constructor / `ngOnInit` and read `console.log`.
- Where to read logs: Safari Web Inspector (iOS WKWebView), `chrome://inspect` (Android WebView), Xcode console / Logcat for the native side.

A.3 Phase 1 — Verify TripTracker Is Truly Standalone

Definition of done: TripTracker is a self-contained package with no dependency on the app or other plugins, and removing it degrades the app GRACEFULLY (no crash).

Do all three layers — the uninstall test alone is not enough.

A. Static / code-level audit

- Native: TripTracker Swift/Kotlin must not import app classes or other plugins' classes. It ships its own `.podspec` (iOS) and `build.gradle` (Android) declaring its own native deps.
- JS/TS: the app imports ONLY the plugin's public API (`import { TripTracker } from '<pkg>'`) — never reaches into internal paths.
- Permissions / background modes: note which ones live in the app's `Info.plist` / `AndroidManifest` (Always-location usage strings, background modes) vs. contributed by the plugin. Usage strings in the app plist are an accepted app-level requirement, not a bug — but document them so integration stays reproducible.

B. Runtime removal test

```
npm uninstall <plugin-package> && npx cap sync
```

(`npx cap sync` triggers `pod install` on iOS.)

Expected: every screen loads and works, no crash.

Reality check: after removal, any `TripTracker.xxx()` call resolves to the web stub and throws `UNIMPLEMENTED`. So “no crash” REQUIRES the app to guard its calls: - `Capacitor.isPluginAvailable('TripTracker')` or feature-detect, plus `try/catch`. - No unguarded plugin call in app bootstrap or page constructors. Phase 1 therefore also produces a call-site inventory confirming each site is guarded.

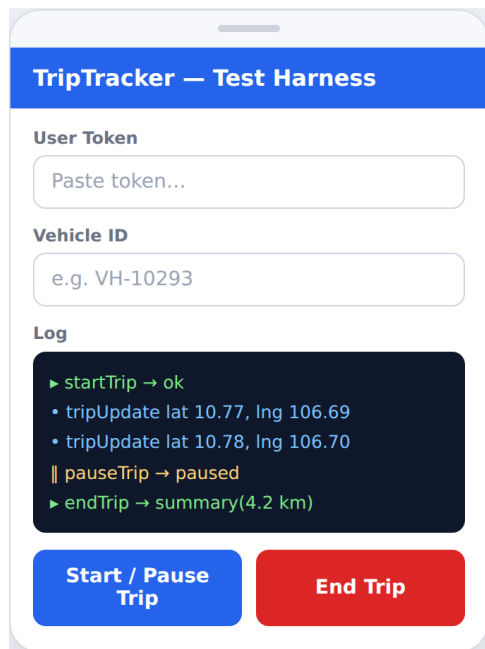
C. Data-contract check

- App reads of shared storage must tolerate a missing key / empty store after removal (no null-deref, no crash).

Deliverable of Phase 1: a short decoupling report — call-site inventory (guarded/unguarded), shared-storage map, and pass/fail of the removal test.

A.4 Phase 2 — Standalone Plugin Test Harness (before integrating into the app)

Suggested UI — TripTracker test harness:



Goal: exercise the released plugin PACKAGE in isolation, so packaging/registration bugs surface before they hit the real app.

Architecture note: build the harness as a MINIMAL CAPACITOR SHELL app that installs the released package (or the packed tarball from `npm pack`) — not a pure-native mini app. Reason: testing a Capacitor plugin release means testing the bridge + auto-registration + podspec/gradle exactly the way the real app consumes them. A pure-native harness bypasses the layer where most release bugs hide (missing export, wrong podspec, plugin not registered). Use a native-only harness only to isolate the native module itself.

Proposed UI (Bảo's layout — kept; customize after Phase 1):

```
User Token: [ _____ ]
Vehicle ID: [ _____ ]

[ log ..... ]
[ log ..... ]
[ log ..... ]
[ log ..... ]
[ log ..... ]

[ Start/Pause Trip ]    [ End Trip ]
```

Wiring: - Start/Pause Trip → `TripTracker.startTrip({ userToken, vehicleId }) / pauseTrip()` - End Trip → `TripTracker.endTrip()` - Log area → append from `addListener('tripUpdate' | 'error' | 'stateChange', ...)` (stream, not a single response) so foreground/background transitions are visible. - Suggested extras to make it a real test rig (optional): show current permission state (Always vs WhenInUse), show foreground/background state, and a “copy logs” action for bug reports.

When Phase 1 is done, Ms. Hieu pings Bảo to finalize the input/button set.